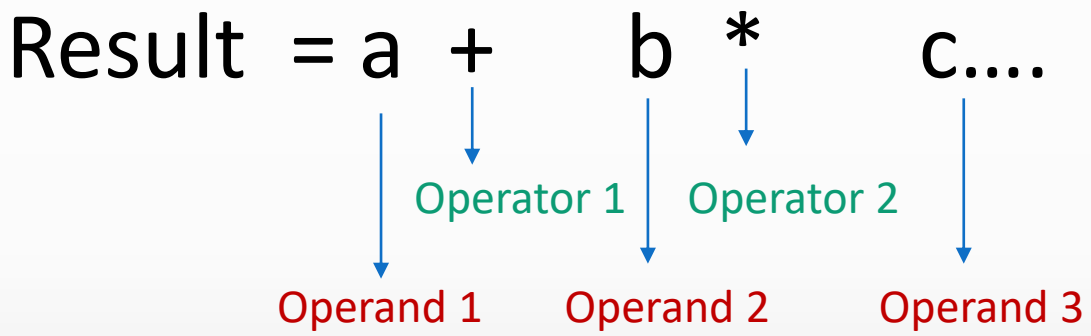
Expression

An expression is a combination of operators, constants and variables. An expression may consist of one or more operands, and zero or more operators to produce a value.



The way to write arithmetic expression is known as a notation. An arithmetic expression can be written in three different but equivalent notations, i.e., without changing the essence or output of an expression.

- Prefix
- Infix
- Postfix

Continue……

Prefix Notation

In this notation, operator is **prefixed** to operands, i.e. operator is written ahead of operands.

For example, **+ab**. This is equivalent to its infix notation **a + b**.

Prefix notation is also known as **Polish Notation**.

Postfix Notation

This notation style is known as **Reversed Polish Notation**.

In this notation style, the operator is **postfixed** to the operands i.e., the operator is written after the operands.

For example, **ab+**. This is equivalent to its infix notation **a + b**.

Infix Notation

Infix notations are normal notations, that are used by us while write different mathematical expressions.

Example, **a+b, a*b+c ,……**

Infix Expression Evaluation

1.Have two stack, one for storing ***operators***, and another one for storing ***operands***.

2. Iterate over the expression from left to right.

3. While iterating, if you get an operand, just push it in the operand stack.

4. While iterating over the expression, if you encounter an operator (say, ***op1***)

- if the operator stack is empty or if the precedence of the operator (op1) is **greater than** the operator on the top of the operator stack, then just push the operator in the operator stack.
- if the operator we encounter has a precedence **less than or equal to** the precedence of the operator that is at the top of the operator stack.

In this case, we need to evaluate the expression first, starting from the last operand encountered, in backward, till we either find a opening brace '(' or an operator which has precedence lower than or equal to the operator op1. To achieve this we need to follow the below three steps.

It is important to note, why we are doing the above two steps**even when the precedence is same**. The short answer is **because of ASSOCIATIVITY.**

Consider the expression 6 - 3 - 2.
This expression can have two answers (of course, one of them is not correct) depending on how we calculate it.
We get the calculated value as 5 if we calculate the expression in the following way: 6 - (3 - 2). But is this correct? No.
We get the correct answer if we calculate 6 - 3 - 2 as (6 - 3) - 2 and we get 1.

So what we can conclude here is
we need to process (calculate) from left to right if the precedence of the operators are the same.

A little more on this: Addition (+) and Multiplication (*) operators are Associative. If an expression has all associative operators with same precedence, it does not matter in which order you process them you will get the same answer.
3 + 4 + 5 = (3 + 4) + 5 = 3 + (4 + 5) = 12
3 * 4 * 5 = (3 * 4) * 5 = 3 * (4 * 5) = 60.

Now that we have visited all the characters in the expression and put them in the stack and done any operations necessary, **we need to process the operands and operators present in the two stack.**

Important to notice that: for whatever we have left in the two stacks now, NON-ASSOCIATIVITY does not matter right now, what I mean by this you would get same calculated value irrespective of you process the left-over expression we have right now from left to right or right to left.

To do this,

1. Pop the top two operands and pop the operators stack, process and put the result in the operands stack.
2. Repeat till the operators stack is empty.

2*(5*(3+6))/15-2

Token	Action	Operator Stack	Operand Stack	Remark
2	Push(2) into operand stack		2	
*	Push into operator stack	*	2	
(	Push into operator stack	(*	2	
5	Push(5) into operand stack	(*	5 2	
*	Push into operator stack	* (*	5 2	
(	Push into operator stack	(* (*	5 2	
3	Push(3) into operand stack	(* (*	3 5 2	
+	Push into operator stack	+ (* (*	3 5 2	
6	Push(6) into operand stack	+ (* (*	6 3 5 2	
)	Pop(6) and pop (3) from operand stack and + from operator stack Do 3+6= 9 Push (9) into operand stack and remove (from operator stack	(* (*	9 5 2	Do process until (is popped from operator stack

2*(5*(3+6))/15-2

Token	Action	Operator Stack	Operand Stack	Remark
)	Pop(6) and pop (3) from operand stack and + from operator stack Do 6+3= 9 Push (9) into operand stack and remove (from operator stack	* (*	9 5 2	
)	Pop(9) and pop (5) from operand stack and * from operator stack Do 5*9=45 Push (45) into operand stack and remove (from operator stack	*	45 2	
/	Push into operator stack	/ *	45 2	
15	Push(15) into operand stack	/ *	15 45 2	
-	Pop(15) and pop (45) from operand stack and / from operator stack Do 45/15=3 Push (3) into operand stack Pop(3) and pop (2) from operand stack and * from operator stack	*	3 2	- has lower precedence than / do process

2*(5*(3+6))/15-2

Token	Action	Operator Stack	Operand Stack	Remark
-	Pop(15) and pop (45) from operand stack and / from operator stack Do 45/15=3 Push (3) into operand stack Pop(3) and pop (2) from operand stack and * from operator stack Do 2*3= 6 Push (6) into operand stack Push – into operator stack	* -	3 2 6	- has lower precedence than / do process has lower precedence than * do process
2	Push(2) into operand stack	-	2 6	
	Pop(2) and (6) from operand stack Pop (-) from operator stack Do 6-2=4		4	Given expression is iterated, do process till operator stack is not empty, it will give the final result.
	Push (4) into operand stack			

Prefix Expression Evaluation

- Reverse the postfix expression.
- Create an operand stack.
- If the character is an operand, push it to the operand stack.
- If the character is an operator, pop two operands from the stack, operate and push the result back to the stack.
- After the entire expression has been traversed, pop the final result from the stack.

- / * 2 * 5 + 3 6 5 2

2 5 6 3 + 5 * 2 * / -

Token	Action	Stack
2	Push(2)in operand stack	2
5	Push(5)in operand stack	5 2
6	Push(6)in operand stack	6 5 2
3	Push(3)in operand stack	3 6 5 2
+	Pop(3) and Pop(6) preform 3+6=9 Push(9) in operand stack	9 5 2
5	Push(5)in operand stack	5 9 5 2
*	Pop(5) and Pop(9) preform 5*9=45 Push(45) in operand stack	45 5 2

2 5 6 3 + 5 * 2 * / -

Token	Action	Stack
*	Pop(5) and Pop(9) preform 5*9=45 Push(45) in operand stack	45 5 2
2	Push(2)in operand stack	2 45 5 2
*	Pop(2) and Pop(45) preform 2*45=90 Push(90) in operand stack	90 5 2
/	Pop(90) and Pop(5) preform 90/5=45 Push(45) in operand stack	45 2
-	Pop(45) and Pop(2) preform 45-2=43 Push(43) in operand stack	43

Postfix Expression Evaluation

- Create a stack to store operands (or values).
- Scan the given expression from left to right and do the following for every scanned element.
  - If the element is a number, push it into the stack.
  - If the element is an operator, pop operands for the operator from the stack. Evaluate the operator and push the result back to the stack.
- When the expression is ended, the number in the stack is the final answer.

2 3 1 * + 9 -

Token	Action	Stack
2	Push(2)in operand stack	2
3	Push(3)in operand stack	3 2
1	Push(1)in operand stack	1 3 2
*	Pop(1) and Pop(3) preform 3*1=3 Push(3) in operand stack	3 2
+	Pop(3) and Pop(2) preform 2+3=5 Push(5) in operand stack	5
9	Push(9)in operand stack	9 5
-	Pop(9) and Pop(5) preform 5-9=-4 Push(-4) in operand stack	- 4

Conversion

- Prefix to Infix Conversion
- Postfix to Infix Conversion
- Infix to Prefix Conversion
- Infix to Postfix Conversion

Conversion – Postfix to Infix

- Start Iterating the given Postfix Expression from Left to right
- If Character is operand then push it into the stack.
- If Character is an operator then pop top 2 Characters which is operands from the stack.
- After poping create a string in which coming operator will be in between the operands.
- Push this newly created string into the stack.
- The above process will continue till the expression have characters left
- At the end only one value will remain if there are integers in expressions. If there is a character then one string will be in output as infix expression.

Example: abc-+de-+

Token	Stack	Action
a	a	Push (a)
b	a b	Push (b)
c	a b c	Push (c)
-	a (b-c)	Pop (b) and Pop(c) and place – in between and push(b-c) into the stack
+	(a+(b-c))	Pop (a) and Pop(b-c) and place + in between and push(a+(b-c)) into the stack
d	(a+(b-c)) d	Push(d)
e	(a+(b-c)) d e	Push (e)
-	(a+(b-c)) (d-e)	Pop (e) and Pop(d) and place - in between and push (d-e)into the stack
+	(a+(b-c))-(d-e)	Pop (d-e) and Pop(a+(b-c)) and place + in between and push (a+b-c))+(d-e)into the stack

operand 2<operator> operand 1

Conversion – Prefix to Infix

- Start scanning the prefix expression from right to left.
- If the current character is an operand (a number or a variable), push it onto a stack.
- If the current character is an operator (+, -, *, /, ^), pop two operands from the stack and concatenates them with the operator in between to form an infix sub-expression. Then push the infix sub-expression onto the stack.
- Repeat steps 2 and 3 until the entire prefix expression has been scanned.
- The final infix expression will be the only item remaining on the stack.

*-a/bc-/ade

Token	Stack	Action
e	e	Push(e)
d	d e	Push(d)
a	a d e	Push(a)
/	(a/d) e	Pop (a) and Pop(b) and place / in between and push(a/d) into the stack
-	((a/d)-e)	Pop (a/d) and Pop(e) and place - in between and push((a/d)-e) into the stack
c	c ((a/d)-e)	Push(c)

operand 1<operator> operan

*-a/bc-/ade

Token	Stack	Action
c	c ((a/d)-e)	Push(c)
b	b c ((a/d)-e)	Push(b)
/	(b/c) ((a/d)-e))	Pop (b) and Pop(c)and place / in between and push(b/c) into the stack
a	a (b/c) ((a/d)-e))	Push(a)
-	(a-(b/c)) ((a/d)-e))	Pop (a) and Pop(b/c) and place - in between and push(a-(b/c))into the stack
*	((a-(b/c)) * ((a/d)-e)))	Pop (a-(b/c)) and Pop((a/d)-e)) and place *in between and push(a-(b/c)) * ((a/d)-e)) into the stack

Infix to Prefix Conversion

- First, reverse the infix expression given in the problem.
- Scan the expression from left to right.
- Whenever the operands arrive, print them.
- If the operator arrives and the stack is found to be empty, then simply push the operator into the stack.
- If the incoming operator has higher precedence than the TOP of the stack, push the incoming operator into the stack.
- If the incoming operator has the same precedence with a TOP of the stack, push the incoming operator into the stack.
- If the incoming operator has lower precedence than the TOP of the stack, pop, and print the top of the stack. Test the incoming operator against the top of the stack again and pop the operator from the stack till it finds the operator of a lower precedence or same precedence.
- If the incoming operator has the same precedence with the top of the stack and the incoming operator is ^, then pop the top of the stack till the condition is true. If the condition is not true, push the ^ operator.

- When we reach the end of the expression, pop, and print all the operators from the top of the stack.
- If the operator is ')', then push it into the stack.
- If the operator is '(', then pop all the operators from the stack till it finds) opening bracket in the stack.
- If the top of the stack is ')', push the operator on the stack.
- At the end, reverse the output.

K+L-M*N+(O^P)*W/U/V*T+Q

Q+T*V/U/W*)P^O(+N*M-L+K

Token	Action	Operator Stack	Operand Stack	Remark
Q	Push(Q) into operand stack		Q	
+	Push(+) into operator stack	+	Q	
T	Push(T) into operand stack	+	Q T	
*	Push(*) into operator stack	+ *	Q T	
V	Push(V) into operand stack	+ *	Q T V	
/	Push(/) into operator stack	+ * /	Q T V	
U	Push(U) into operand stack	+ * /	Q T V U	
/	Push(/) into operator stack	+ * / /	Q T V U	
W	Push(W) into operand stack	+ * / /	Q T V U W	
*	Push(*) into operator stack	+ * / / *	Q T V U W	

Q+T*V/U/W*)P^O(+N*M-L+K

Token	Action	Operator Stack	Operand Stack	Remark
*	Push(*) into operator stack	+ * / / *	Q T V U W	
)	Push()) into operator stack	+ * / / *)	Q T V U W	
P	Push(P) into operand stack	+ * / / *)	Q T V U W P	
^	Push(^) into operator stack	+ * / / *) ^	Q T V U W P	
O	Push(O) into operand stack	+ * / / *) ^	Q T V U W P O	
(	Push(() into operator stack	+ * / / *	Q T V U W P O ^	
+	Push(+) into operator stack	+ +	Q T V U W P O ^ * / / *	+ Lower
N	Push(N) into operand stack	+ +	Q T V U W P O ^ * / / * N	
*	Push(*) into operator stack	+ + *	Q T V U W P O ^ * / / * N	
M	Push(M) into operand stack	+ + *	Q T V U W P O ^ * / / * N M	

Q+T*V/U/W*)P^O(+N*M-L+K

Token	Action	Operator Stack	Operand Stack	Remark
M	Push(M) into operand stack	+ + *	Q T V U W P O ^ * / / * N M	
-	Push(-) into operator stack	+ + -	Q T V U W P O ^ * / / * N M *	
L	Push(L) into operand stack	+ + -	Q T V U W P O ^ * / / *N M * L	
+	Push(+) into operator stack	+ + - +	Q T V U W P O ^ * / / *N M * L	
K	Push(K) into operand stack	+ + - +	Q T V U W P O ^ * / / * N M * L	K + + -+

Q T V U W P O ^ * / / * N M * L K +- ++

+ + - + K L * M N */ / * ^ O P W U V T Q

Infix to Postfix Conversion

- Scan the infix expression from left to right.
- If the scanned character is an operand, put it in the postfix expression.
- Otherwise, do the following
    - ❖ If the precedence and associativity of the scanned operator are greater than the precedence and associativity of the operator in the stack [or the stack is empty or the stack contains a ‘(‘], then push it in the stack. [‘^‘ operator is right associative and other operators like ‘+‘,’–‘,’*‘ and ‘/‘ are left-associative].
        - ➢ Check especially for a condition when the operator at the top of the stack and the scanned operator both are ‘^‘. In this condition, the precedence of the scanned operator is higher due to its right associativity. So it will be pushed into the operator stack.
        - ➢ In all the other cases when the top of the operator stack is the same as the scanned operator, then pop the operator from the stack because of left associativity due to which the scanned operator has less precedence.

- ❖ Else, Pop all the operators from the stack which are greater than or equal to in precedence than that of the scanned operator.
    - ➢ After doing that Push the scanned operator to the stack. (If you encounter parenthesis while popping then stop there and push the scanned operator in the stack.)
- If the scanned character is a ‘(‘, push it to the stack.
- If the scanned character is a ‘)’, pop the stack and output it until a ‘(‘ is encountered, and discard both the parenthesis.
- Repeat steps 2-5 until the infix expression is scanned.
- Once the scanning is over, Pop the stack and add the operators in the postfix expression until it is not empty.
- Finally, print the postfix expression.

K+L-M*N+(O^P)*W/U/V*T+Q

Token	Action	Operator Stack	Operand Stack	Remark
K	Push(K) into operand stack		K	
+	Push(+) into operator stack	+	K	
L	Push(L) into operand stack	+	K L	
-	Push(-) into operator stack	-	K L +	
M	Push(M) into operand stack	-	K L + M	
*	Push(*) into operator stack	- *	K L + M	
N	Push(N) into operand stack	- *	k L + M N	
+	Push(+) into operator stack	+	K L + M N * -	
(	Push(() into operand stack	+ (	K L + M N * -	
O	Push(O) into operator stack	+ (	K L + M N * - O	

K+L-M*N+(O^P)*W/U/V*T+Q

Token	Action	Operator Stack	Operand Stack	Remark
O	Push(O) into operator stack	+ (	K L + M N * - O	
^	Push(^) into operator stack	+ (^	K L + M N * - O	
P	Push(P) into operand stack	+ (^	K L + M N * - O P	
)	Pop ^ from operator stack	+	K L + M N * - OP ^	
*	Push(*) into operator stack	+ *	K L + M N * - OP ^	
W	Push(W) into operand stack	+ *	K L + M N * - OP ^ W	
/	Push(/) into operator stack	+ /	K L + M N * - O P ^ W *	
U	Push(U) into operand stack	+ /	K L + M N * - O P ^ W * U /	
/	Push(/) into operator stack	+ /	K L + M N * - OP ^ W *U /	
V	Push(V) into operand stack	+ /	K L + M N * - OP ^ W *U / V	

K+L-M*N+(O^P)*W/U/V*T+Q

Token	Action	Operator Stack	Operand Stack	Remark
V	Push(V) into operand stack	+ /	K L + M N * - OP ^ W *U / V	
*	Push(*) into operator stack	+ *	K L + M N * - O P ^ W *U / V /	
T	Push(T) into operand stack	+ *	K L + M N * - O P ^ W *U / V / T	
+	Push(+) into operator stack	+	K L + M N * - O P ^ W *U / V / T * +	
Q	Push(Q) into operand stack	+	K L + M N * - O P ^ W *U / V / T* + Q	
		-	K L + M N * - O P ^ W *U / V / T * + Q +	
	K L + M N * - O P ^ W *U / V / T * + Q +			

Problems

abc/-ad/e-*

2+3*5+50/5^2